Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего образования «Омский государственный технический университет»
Кафедра «Автоматизированные системы обработки информации и управления»

ОТЧЕТ
ПО ЛАБОРАТОРНЫМ РАБОТАМ № 1 и № 2

по дисциплине «Кроссплатформенная разработка»
студента Абрамова Артёма Евгеньевича группы ПИН-252т

Пояснительная записка
Шифр работы От-43-ПИН-252т-1 ЛР
Специальность 09.03.04 Программная инженерия

Старший преподаватель М.А. Мищенко

Студент А.Е. Абрамов

Омск 2026

РЕФЕРАТ

22 страницы, 5 рисунков, 2 таблицы, 10 листингов

Объектом выполненной работы является каркас кроссплатформенного приложения «Каталог видов», разработанный на разных технологических стеках: Compose Multiplatform (Kotlin) и Flutter (Dart).

Целью работы является построение каркаса (архитектурной основы) приложения: навигации между экранами, экрана-списка на моковых данных, переключения темы и локализации интерфейса, а также перехода по иерархии таксонов.

В работе применены архитектурный подход с разделением на слои domain, data и presentation/ui, однонаправленный поток данных (состояние и намерения) и библиотеки навигации с типизированными маршрутами.

Степень внедрения - для учебных целей.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 4
1 Постановка задачи 5
2 Архитектура приложения 7
3 Реализация 9
3.1 Модели предметной области и репозиторий 9
3.2 Моковые данные 10
3.3 Состояние экрана и намерения 11
3.4 Навигация 13
3.5 Тема 15
3.6 Локализация 15
3.7 Поиск по названию таксона 17
4 Проверка работы 18
ЗАКЛЮЧЕНИЕ 21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 22

ВВЕДЕНИЕ

Кроссплатформенная разработка позволяет описать интерфейс и логику приложения один раз и собрать их под несколько платформ. В рамках курса были изучены два подхода к созданию каркаса: Compose Multiplatform (KMP) - использует язык Kotlin и Flutter - использует язык Dart.

Предметной областью приложения выбран каталог биологических видов. Источник данных - открытый программный интерфейс GBIF Species API, который отдаёт сведения о таксонах: научное название, ранг, родительский таксон, число потомков. На первой вехе курса сеть не используется: приложение работает на моковых данных, повторяющих форму ответа сервера, поэтому на следующей вехе моки заменяются сетевым источником без изменения остального кода.

Цель работы - разработать каркас кроссплатформенного приложения, то есть такую основу, которая не рассыпается при обращении с ней: при переключении темы, смене языка, переходе на экран детали и возврате назад, и сравнить, как одни и те же архитектурные паттерны реализуются в Compose и Flutter.

Для достижения цели поставлены следующие задачи:

— создать проекты на базе Compose Multiplatform и Flutter с таргетами для веба и настольных систем;

— разложить код по слоям domain, data и presentation так, чтобы зависимости сходились к слою предметной области;

— подготовить моковый набор данных не менее чем из двадцати таксонов по форме программного интерфейса;

— реализовать экран-список, экран детали таксона и экран дочерних таксонов, позволяющий ходить по дереву любой глубины;

— организовать навигацию на библиотеках Navigation 3 и go_router;

— вынести цвета в тему и реализовать переключение светлого и тёмного оформления;

— вынести все подписи интерфейса в строковые ресурсы и добавить вторую локаль;

— реализовать поиск по названию таксона;

— провести сравнительный анализ реализации каркаса на двух стеках.

1 Постановка задачи

Приложение представляет собой каталог биологических видов и состоит из трёх экранов: списка таксонов, детали таксона и списка дочерних таксонов. Список показывает научное название таксона, его номер и бейдж ранга. Деталь содержит сведения, которых нет в списке: полное научное название, описание таксона и кнопку перехода к дочерним таксонам. С детали доступен переход к дочерним таксонам, а с них - к их потомкам, что позволяет ходить по дереву любой глубины.

Источником данных на последующих вехах выступает программный интерфейс GBIF Species API по адресу https://api.gbif.org/v1/species/search. Он не требует ключа, поддерживает постраничную выдачу параметрами limit и offset и содержит около сорока восьми миллионов записей. Ограничение источника - rate limit на быстрые запросы, поэтому данные для моков снимались с обычного эндпоинта без параметров и содержат первые двадцать записей ответа.

Требования к каркасу, проверяемые при сдаче работы, приведены в таблице 1.

Таблица 1 — Требования к каркасу приложения

Требование	**Способ выполнения на Flutter**	**Способ выполнения на KMP**
Навигация, маршрут - объект	Использован go_router с вложенными маршрутами и строгими путями (/taxon/:key)	Маршруты описаны sealed-интерфейсом, аргумент лежит полем объекта (Navigation 3)
Возврат выглядит как возврат	В CustomTransitionPage задана прямая и обратная анимация слайда	Заданы все три спецификации переходов, включая обратный и предиктивный
Подписи в строковых ресурсах, две локали	Каталоги l10n/ с ARB-файлами, паритет проверяется скриптом	Каталоги values и values-ru, паритет ключей проверяется скриптом на Python
Тема из токенов оформления	Цвета объявлены в AppTheme и AppColors, UI читает их через Theme.of(context)	Цвета объявлены только в файле темы, поверхности читают их из схемы MaterialTheme

Продолжение таблицы 1

Экран рисуется из состояния	Используются Cubit с sealed-состояниями, экран получает состояние и колбэк	В функцию экрана передаются состояние и обработчик намерений (ViewModel + StateFlow)
Поиск по названию таксона	Ввод подстроки сужает список, запрос сохраняется при возврате с детали	Ввод подстроки сужает список, запрос сохраняется при возврате с детали

2 Архитектура приложения

Код приложения разделён на три слоя. Слой domain содержит модели предметной области и описание репозитория в виде интерфейса. Слой data содержит моковые данные и реализацию репозитория. Слой presentation в Flutter и ui в KMP содержит экраны, переиспользуемые компоненты, тему, навигацию и модели представления. Все зависимости направлены к слою domain: слой presentation пользуется его моделями, слой data преобразует полученные данные в эти же модели.

Такое разделение решает практическую задачу: на следующей вехе моковый репозиторий заменяется сетевым, и вся правка происходит внутри папки data, а экраны и модели предметной области остаются неизменными.

Структура каталогов общего модуля приведена в листинге 1.

```
// KMP (Compose Multiplatform)
shared/src/commonMain/kotlin/ru/omgtu/abramov/
  App.kt               корень, где собираются тема, репозиторий,
                       навигатор и фабрики
  Screen.kt            маршруты приложения
  domain/              модели и интерфейс репозитория
  data/                моки и реализация репозитория
  utils/               форматирование и фильтрация
  ui/list/             состояние, намерения и ViewModel списка
  ui/AppTheme.kt       светлая и тёмная палитры
  ui/model/            модели представления
  ui/navigation/       навигатор и хост навигации
  ui/detail/           состояние, намерения и ViewModel детали
  ui/screens/          вёрстка экранов
  ui/components/       переиспользуемые элементы интерфейса
shared/src/commonMain/composeResources/
  values/               подписи интерфейса на английском
  values-ru/            подписи интерфейса на русском
  drawable/             иконки

// Flutter
lib/
  main.dart
  app.dart      корень, где собираются тема, локаль, репозиторий
  common/
    navigation/             маршруты приложения(app_router.dart)
    theme/                  палитры и цвета
    widgets/                общие виджеты (AppScaffold)
  features/
    taxon/
      domain/               модели и интерфейс репозитория
      data/                 моки и реализация репозитория
      presentation/
        cubit/              состояние, намерения и Cubit
        utils/              форматирование и фильтрация
        view/               вёрстка экранов и компоненты
```

Листинг 1 - Структура общего модуля

Логика каждого экрана вынесена в отдельный класс (ViewModel в KMP и Cubit во Flutter). Внутри него состояние хранится в неизменяемом поле, а наружу отдаётся поток только для чтения, поэтому изменить состояние извне невозможно. Действия пользователя приходят в единственный метод обработки намерений или напрямую вызывают методы.

Зависимости уровня приложения - репозиторий, навигатор, тема и локаль - создаются в корневой функции приложения и передаются вниз через провайдеры.

3 Реализация

3.1 Модели предметной области и репозиторий

Модели повторяют форму ответа программного интерфейса, включая типы идентификаторов. Фрагмент моделей и описание репозитория в KMP приведены в листинге 2.

```
data class Taxon(
    val key: Int,
    val scientificName: String,
    val canonicalName: String?,
    val rank: Rank,
    val parentKey: Int?,
    val numDescendants: Int,
    val kingdom: String?,
    val phylum: String?,
    val description: String?,
)

enum class Rank {
    KINGDOM,
    PHYLUM,
    CLASS,
    ORDER,
    FAMILY,
    GENUS,
    SPECIES
}

interface TaxonRepository {
    suspend fun getTaxons(): List<Taxon>
    suspend fun getTaxon(key: Int): Taxon?
    suspend fun getChildren(parentKey: Int): List<Taxon>
}
```

Листинг 2 – Модель таксона и интерфейс репозитория в KMP

Фрагмент моделей и описание репозитория в Flutter приведены в листинге 3.

```
final class TaxonModel {
  const TaxonModel({
    required this.key,
    required this.scientificName,
    this.canonicalName,
    required this.rank,
    this.parentKey,
    required this.numDescendants,
    this.kingdom,
    this.phylum,
    this.description,
  });

  final int key;
  final String scientificName;
  final String? canonicalName;
  final TaxonRank rank;
  final int? parentKey;
  final int numDescendants;
  final String? kingdom;
  final String? phylum;
  final String? description;
}

enum TaxonRank {
  kingdom, phylum, klass, order, family, genus, species;
}

abstract interface class ITaxonRepository{
  Future<List<TaxonModel>> getTaxons();
  Future<TaxonModel?> getTaxon(int key);
  Future<List<TaxonModel>> getChildren(int parentKey);
  Future<List<TaxonModel>> search(String query, {int?
parentKey});
}
```

Листинг 3 - Модель таксона и интерфейс репозитория в Flutter

3.2 Моковые данные

Моки содержат двадцать таксонов: девять царств и одиннадцать типов. Записи снимались с программного интерфейса, поэтому названия, ранги и число потомков являются настоящими. Ранги подобраны разными: царство и тип. Фрагмент моков из KMP и Flutter приведён в листинге 4.

```
// KMP
val mockTaxons: List<Taxon> = listOf(
    Taxon(
        key = 0,
        scientificName = "incertae sedis",
        canonicalName = "incertae sedis",
        rank = Rank.KINGDOM,
        parentKey = null,
        numDescendants = 3613,
        kingdom = "incertae sedis",
        phylum = null,
        description = null,
    ),…

// Flutter
const List<TaxonModel> mockTaxons = [
  TaxonModel(
    key: 0,
    scientificName: "Incertae sedis",
    canonicalName: "incertae sedis",
    rank: TaxonRank.kingdom,
    parentKey: null,
    numDescendants: 3613,
    kingdom: "Incertae sedis",
    phylum: null,
    description: null,
  ),…
```

Листинг 4 - Фрагменты моковых данных

3.3 Состояние экрана и намерения

Экран не обрабатывает нажатия самостоятельно. Ему передаются состояние и функция обработки намерений, а решение принимает ViewModel (в KMP) или Cubit (во Flutter). Такой порядок нужен не только для стройности кода: если передать в экран целиком ViewModel/Cubit, компилятор признает её нестабильным типом, начнёт сравнивать по ссылке и перестанет пропускать перерисовку экрана. Реализация приведена в листинге 5.

```
// KMP (ViewModel)
sealed interface TaxonListIntent {
    data class CardClicked(val key: Int) : TaxonListIntent
    data class QueryChanged(val query: String) : TaxonListIntent
}

class TaxonListViewModel(...) : ViewModel() {
    private val _state = MutableStateFlow(TaxonListState())
    val state: StateFlow<TaxonListState> = _state.asStateFlow()
    // ...
    fun onIntent(intent: TaxonListIntent) {
        when (intent) {
            is TaxonListIntent.CardClicked ->
navigator.addToBackStack(Screen.Detail(intent.key))
            // ...
        }
    }
}

// Flutter (Cubit)
sealed class TaxonListState {
  const TaxonListState();
}

final class TaxonListLoading extends TaxonListState {
  const TaxonListLoading();
}

final class TaxonListLoaded extends TaxonListState {
  const TaxonListLoaded({required this.query, required
this.items});
  final String query;
  final List<TaxonModel> items;
}

class TaxonListCubit extends Cubit<TaxonListState> {
  TaxonListCubit(this._repository, {this.parentKey}) :
super(const TaxonListLoading());

  Future<void> load() async {
    emit(const TaxonListLoading());
    final items = await _repository.getTaxons();
    if (isClosed) return;
    emit(TaxonListLoaded(query: '', items: items));
  }
}
```

Листинг 5 - Состояние, намерения и ViewModel/Cubit экрана-списка

3.4 Навигация

В библиотеке Navigation 3 (KMP) стека маршрутов как отдельной сущности нет: стек - это обычный список объектов. В go_router (Flutter) стек управляется самим роутером, но маршруты также описываются декларативно.

В обоих случаях все пути собраны в одном месте, чтобы строки не расходились по проекту. Маршруты вложены друг в друга: список — корень, деталь вложена в список, а дочерние таксоны - в деталь. Благодаря этому стек навигации выстраивается корректно: при открытии детей и нажатии «Назад» пользователь вернется на деталь, а затем в список, а не сразу в корень.

Переходы анимируются с помощью явного задания прямой и обратной анимации, чтобы возврат выглядел как возврат. Реализация на KMP приведена в листинге 5.

```
// KMP (Navigation 3)
sealed interface Screen {
    data object List : Screen
    data class Detail(val key: Int) : Screen
    data class Children(val parentKey: Int, val parentName:
String) : Screen
}

class Navigator {
    private val _navStack: MutableStateFlow<List<Screen>> =
MutableStateFlow(listOf(Screen.List))
    val navStack = _navStack.asStateFlow()

    fun addToBackStack(screen: Screen) {
        _navStack.update { it + screen }
    }

    fun back() {
        _navStack.update { currentStack ->
            if (currentStack.size > 1) currentStack.dropLast(1)
else currentStack
        }
    }
}
```

Листинг 5 - Маршруты и навигатор в KMP

Реализация на KMP приведена в листинге 6.

```
//Flutter (go_router)
abstract final class AppRoutes {
  static const String list = '/';
  static String detail(int key) => '/taxon/$key';
  static String children(int parentKey) =>
'/taxon/$parentKey/children';
}
abstract final class AppRouter {
  static const Duration _transition = Duration(milliseconds: 300);
  static final GoRouter router = _create();
  static GoRouter _create() {
    GoRouter.optionURLReflectsImperativeAPIs = true;
    return GoRouter(
      initialLocation: AppRoutes.list,
      routes: [
        GoRoute(
          path: AppRoutes.list,
          pageBuilder: (context, state) =>
              _page(state, const TaxonListScreen()),
          routes: [
            GoRoute(
              path: 'taxon/:key',
              pageBuilder: (context, state) {
                final key =
int.tryParse(state.pathParameters['key']!);
                return _page(
                  state,
                  key == null
                      ? const _NotFoundScreen()
                      : TaxonDetailScreen(taxonKey: key),);
              },
              routes: [
                GoRoute(
                  path: 'children',
                  pageBuilder: (context, state) {
                    final key =
int.tryParse(state.pathParameters['key']!);
                    return _page(
                      state,
                      key == null
                         ? const _NotFoundScreen()
                         : TaxonChildrenScreen(parentKey: key));
                  },
                ),
              ],
            ),
          ],
        ),
      ],
      errorBuilder: (context, state) => const _NotFoundScreen(),
    );
  }
```

Листинг 6 - Маршруты и навигатор в Flutter

Классу ViewModel (или Cubit) детали задан ключ, составленный из идентификатора таксона. Без ключа обращение к ViewModel/Cubit возвращает один экземпляр на весь хост навигации, и открытая повторно деталь показала бы предыдущую запись.

3.5 Тема

Тема представляет собой набор токенов оформления, который провайдер кладёт в неявный контекст композиции (в KMP) или в InheritedWidget (во Flutter). Чтение токена является чтением состояния, поэтому при смене темы перерисовываются именно те элементы, которые её прочитали. Цвет, записанный числом прямо в коде элемента, в переключении темы не участвует. По этой причине все цвета объявлены в одном файле.

```
// KMP
private val LightColors = lightColorScheme(
    primary = Color(0xFF2E7D32),
    onPrimary = Color(0xFFFFFFFF),
    surface = Color(0xFFFFFBFE),
    onSurface = Color(0xFF1C1B1F),
    background = Color(0xFFFFFBFE),
    onBackground = Color(0xFF1C1B1F),
)

private val DarkColors = darkColorScheme(
    primary = Color(0xFF81C784),
    surface = Color(0xFF1C1B1F),
    background = Color(0xFF141218),
)

// Flutter
abstract final class AppColors {
  static const Color seed = Color(0xFF008000);

  static const Color rankKingdomBg = Color(0xFF6A1B9A);
  static const Color rankKingdomFg = Colors.white;

  static const Color rankPhylumBg = Color(0xFF4527A0);
  static const Color rankPhylumFg = Colors.white;

  …

  static const Color rankSpeciesBg = Color(0xFFEF6C00);
  static const Color rankSpeciesFg = Colors.white;
}
```

Листинг 7 - Палитры и цвет статуса запуска

3.6 Локализация

Все подписи интерфейса вынесены в строковые ресурсы. В KMP это XML-файлы в composeResources, во Flutter - ARB-файлы. По ним

генерируется класс доступа, поэтому в коде стоит не строковое имя, а сгенерированный идентификатор, и опечатка не компилируется. Квалификатор локали разрешается во время выполнения: один и тот же идентификатор даёт разный текст. На листинге 8 показаны объявления строковых ресурсов в KMP и Flutter

```
// KMP
<!-- values/strings.xml -->
<string name="app_title">Species Catalog</string>
<string name="section_children">Children</string>
<string name="action_back">Back</string>
<string name="action_toggle_theme">Toggle theme</string>

<!-- values-ru/strings.xml -->
<string name="app_title">Каталог видов</string>
<string name="section_children">Дочерние таксоны</string>
<string name="action_back">Назад</string>
<string name="action_toggle_theme">Сменить тему</string>

// Flutter
{
  "@@locale": "en",
  "appTitle": "Species Catalog",
  "actionToggleTheme": "Toggle theme",
  "taxonNumber": "No. {number}",
  "actionBack" : "Back",
  "actionToggleLanguage" : "Switch language",
  "rankKingdom" : "Kingdom",
  "rankPhylum" : "Phylum",
   …
}
{
  "@@locale": "ru",
  "appTitle": "Каталог видов",
  "actionToggleTheme": "Переключить тему",
  "taxonNumber": "No. {number}",
  "actionBack" : "Назад",
  "actionToggleLanguage" : "Переключить язык",
  "rankKingdom" : "Царство",
  "rankPhylum" : "Тип",
  …

}
```

Листинг 8 - Строковые ресурсы

Паритет ключей между локалями проверяется скриптом (check-strings.py в KMP и check_arb.dart во Flutter). Локализуется интерфейс, а не данные

каталога: научные названия таксонов и описания остаются на языке источника.

3.7 Поиск по названию таксона

Поиск реализован так, чтобы в следующей вехе его можно было перенести на сервер GBIF. В KMP поиск реализован через расширение репозитория filterByName. Во Flutter поиск перенесен в метод репозитория search(query, {parentKey}). Ввод подстроки сужает список, регистронезависимо, по научному и каноническому названиям.

При возврате с экрана детали запрос сохраняется: состояние хранится в ViewModel/Cubit, который живёт в корне приложения и не уничтожается при навигации. При отсутствии совпадений экран показывает сообщение «Ничего не найдено» (листинг 9).

```
// KMP

suspend fun TaxonRepository.getTaxons(filter: String?):
List<Taxon> =
    getTaxons().filterByName(filter)

fun List<Taxon>.filterByName(filter: String?): List<Taxon> {
    val needle = filter?.trim().orEmpty()
    if (needle.isEmpty()) return this
    return filter { it.scientificName.contains(needle,
ignoreCase = true) }
}

// Flutter

Future<List<TaxonModel>> search(String query, {int? parentKey})
async {
    final base = parentKey == null
      ? mockTaxons
      : mockTaxons.where((t) => t.parentKey ==
parentKey).toList();

    final needle = query.trim().toLowerCase();
    if (needle.isEmpty) return base;

    return base.where((t) =>
      t.scientificName.toLowerCase().contains(needle) ||
(t.canonicalName?.toLowerCase().contains(needle) ?? false)
    ).toList();
  }
```

Листинг 9 – Реализация поиска

4 Проверка работы

Каркас собирается и запускается на всех таргетах проектов. Команды запуска приведены в листинге 10.

```
// KMP
./gradlew :desktopApp:run - запуск на десктоп
./gradlew :webApp:wasmJsBrowserDevelopmentRun - запуск на
браузер
./gradlew :androidApp:assembleDebug - запуск на андроид
python3 tools/check-strings.py - проверка локализации

// Flutter
flutter run -d edge       # запуск в браузере
flutter run -d windows      # запуск на десктопе
dart run tool/check_arb.dart # проверка локализации
```

Листинг 10 - Команды сборки и проверки

Ведущим таргетом первой вехи выбран веб, зачётными таргетами курса - настольная система, веб и Android.

Порядок приёмки работы приведён в таблице 2.

Таблица 2 — Порядок проверки работы

Действие	**Ожидаемый результат**
Запустить приложение и прокрутить список	Двадцать карточек, у каждой название, номер и бейдж ранга
Открыть карточку и нажать возврат	Деталь выезжает справа, возврат уводит экран вправо, список на месте
На детали открыть дочерние таксоны	Открывается список детей, возврат ведёт на деталь, а не в список. У таксона без детей пустой список
Ввести подстроку в поиск	Список сужается; возврат с детали сохраняет запрос. При несуществующем запросе показано «Ничего не найдено»
Нажать переключатель темы в заголовке	Темнеют заголовок, фон и карточки, подписи остаются читаемыми
Выполнить сценарий проверки локалей	Расхождений ключей нет, выводится количество проверенных ключей
Переключить язык в заголовке	Интерфейс на втором языке, данные каталога остаются на языке источника

Внешний вид приложения приведён на рисунках 1–3. Слева приложение в браузере на KMP, справа – в браузере на Flutter.

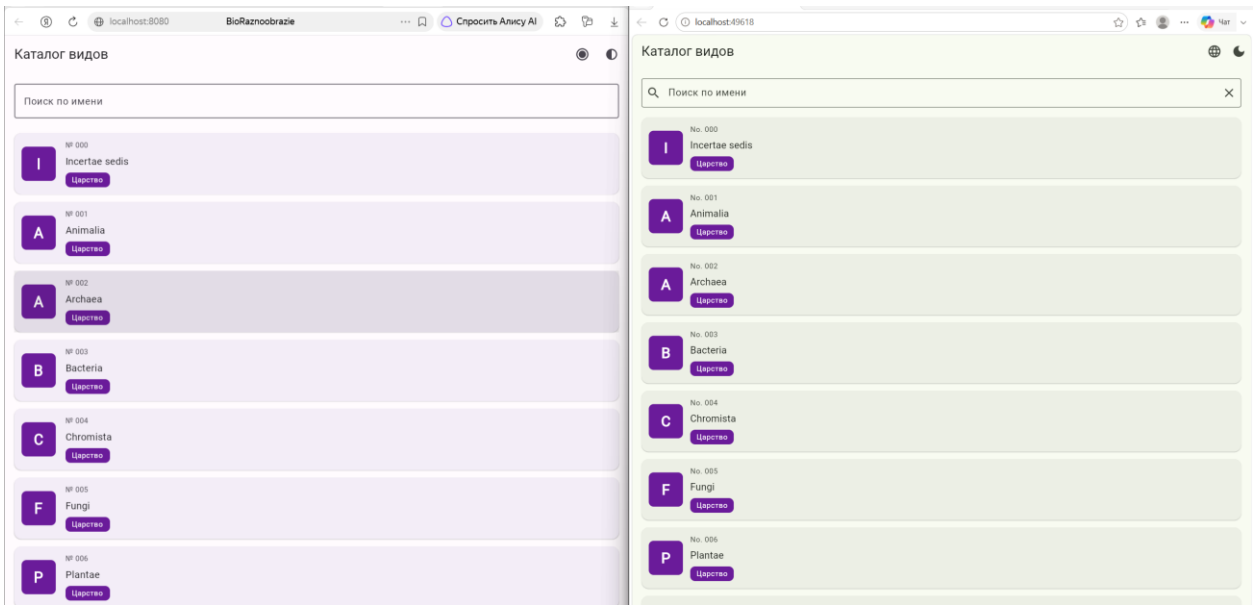


Рисунок 1 - Экран-список таксонов

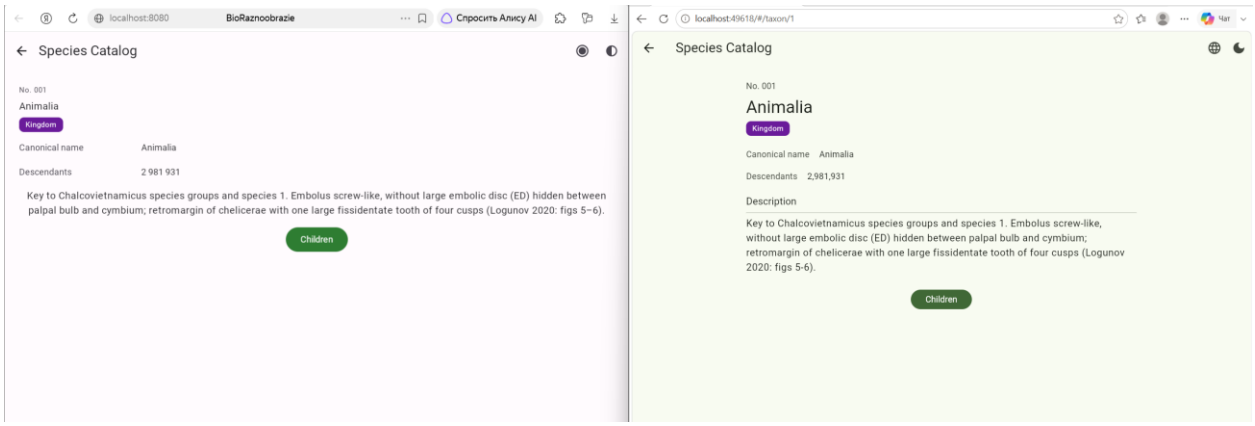


Рисунок 2 - Экран детали таксона с другим языком

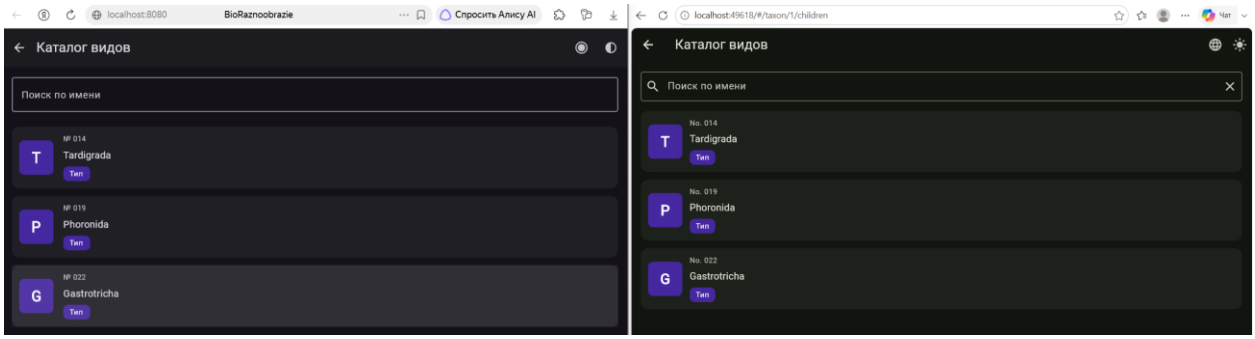


Рисунок 3 - Экран дочерних таксонов с тёмной темой

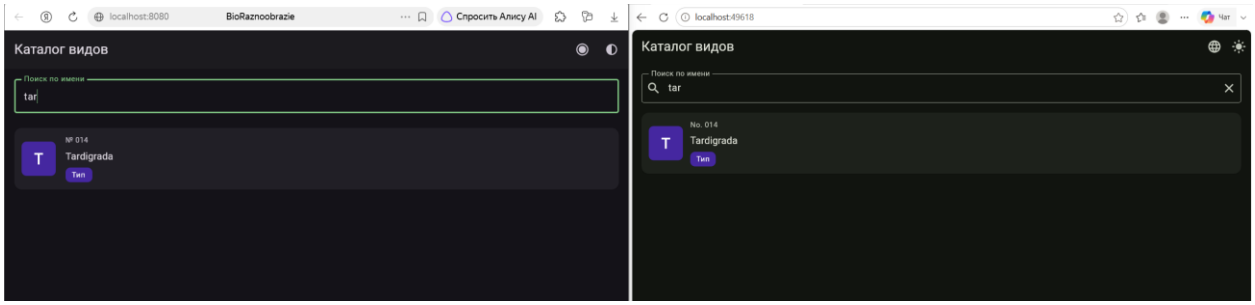


Рисунок 4 – Поиск по названию таксона

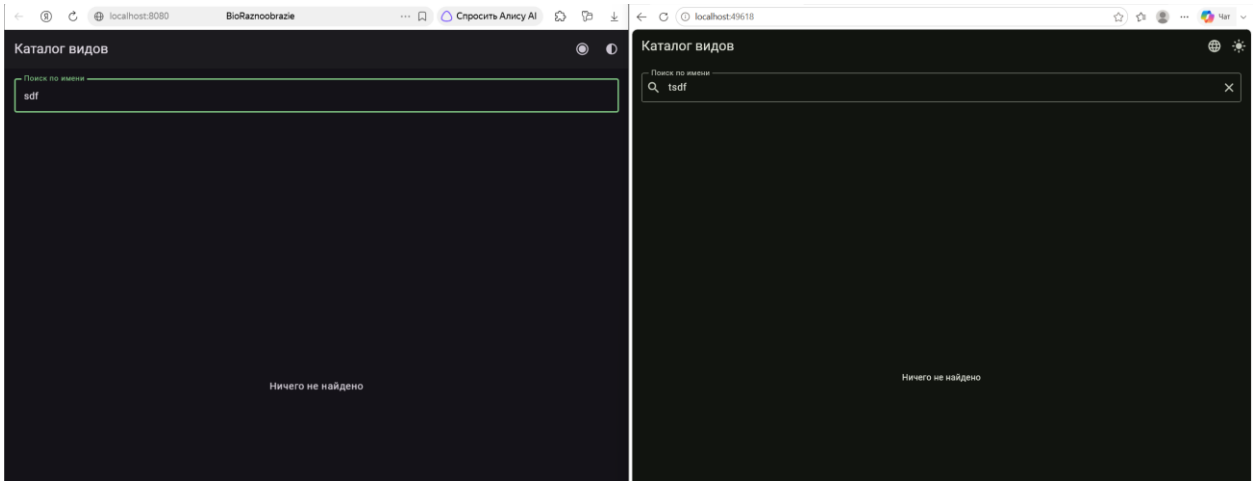


Рисунок 5 – Попытка поиска несуществующей записи

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторных работ поставленная цель достигнута: разработан каркас кроссплатформенного приложения «Каталог видов» на двух технологических стеках: Compose Multiplatform и Flutter. Все поставленные задачи выполнены.

Код разложен по слоям domain, data и presentation, зависимости направлены к слою предметной области, а моковый набор из двадцати таксонов повторяет форму программного интерфейса GBIF Species API. Благодаря этому замена моков на сетевой источник на следующей вехе затронет только слой data.

Реализованы три экрана и навигация между ними (Navigation 3 для KMP и go_router для Flutter). Экраны получают состояние и обработчик намерений, а не класс ViewModel/Cubit целиком. Цвета вынесены в тему, переключение светлого и тёмного оформления работает, подписи интерфейса вынесены в строковые ресурсы и переведены на второй язык. Поиск по названию таксона работает и сохраняет запрос при возврате с экрана детали.

Практическая ценность работы состоит в полученном опыте разделения приложения на слои и в понимании того, что архитектура каркаса не зависит от фреймворка. Полученный каркас служит основой для следующей вехи курса, на которой будут добавлены работа с сетью и постраничная загрузка.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1) Flutter: официальная документация [Электронный ресурс]. — URL: https://docs.flutter.dev/

2) go_router: официальная документация [Электронный ресурс]. — URL: https://pub.dev/packages/go_router

3) flutter_bloc: официальная документация [Электронный ресурс]. — URL: https://pub.dev/packages/flutter_bloc

4) Compose Multiplatform: официальная документация [Электронный ресурс]. — URL: https://www.jetbrains.com/help/kotlin-multiplatform-dev/compose-multiplatform.html

5) Navigation 3: официальная документация [Электронный ресурс]. — URL: https://developer.android.com/guide/navigation/navigation-3

6) Material Design 3: цветовые схемы и токены [Электронный ресурс]. — URL: https://m3.material.io/styles/color/system